SPVS setup and demonstration guide

This guide accompanies the source ZIP. The complete design and measured results are in the project report and verification report.

1. Unpack and inspect

Extract `SPVS_COMPLETE_PROJECT.zip`. Its `spvs-root` directory contains actual application files, migrations, tests, lockfiles and documentation. It excludes `.env`, installed dependencies, biometric data and database volumes. The included model licenses describe the official pretrained assets. Read `SPVS_DELIVERY_README.md` for the current validation boundary.

2. Prepare the development machine

Install Docker Desktop, Python 3.12, uv, Node 24 LTS and Git. On an M4 Mac use ARM64 installations. Run these commands from `spvs-root`:

```
make setup
make db
make migrate
make seed
```

Setup generates secrets only when `.env` does not exist. Keep the existing file across restarts. The demo accounts are `admin@spvs.example`, `faculty@spvs.example` and `student@spvs.example`. Read `DEMO_PASSWORD` locally from `.env` when signing in. Do not send it to an external service or include it in screenshots.

3. Start desktop components

Run `make api` in one terminal and `make dashboard` in another. The API uses port 8000 and web dashboard 5173. Check `/health` for database readiness and `/ready` for model initialization. The API's interactive contract is at `/docs`. In the administrator dashboard, configure real classroom coordinates and Wi-Fi references or select an explicit fallback policy.

4. Prepare the student phone

Install Android Studio and compatible SDK tools. In `mobile`, copy `.env.example` to `.env`, set the API URL ending in `/api/v1`, then run `npm ci` and `npm run android`. A physical phone needs the developer computer's reachable LAN address or trusted HTTPS URL. Expo Go is insufficient for the local Wi-Fi module. iOS requires Xcode/signing and reports Wi-Fi scan support as unavailable.

Grant camera and foreground location permissions. Register the phone using password confirmation, then consent to enrollment. Use a single face, good lighting and a slow blink. Do not use fake GPS, a generated portrait or preapproved attendance to make a demo appear successful.

5. Validate before evaluation

Create the separate test database, fetch public smoke-test samples and run `make test` and `make build`. Start the API/dashboard and run the browser tests. Read the current generated results rather than assuming a zero exit code means hardware validation. Record physical-device cases separately.

6. Demonstrate

Use the sequence in `docs/DEMO_SCRIPT.md`: academic configuration, session creation, real student verification when hardware is validated, understandable decision, review/correction audit, session closure and export. Explain uncertainty and operating-system limits. Show an empty chart honestly when there have been no attempts.

7. Recover and maintain

Restart the existing database and application processes without deleting volumes. If secrets are lost, restore them from the operator's protected backup. Do not disable TLS or checksum verification for a failed download. Use the deployment troubleshooting table and the exact failure reasons returned by the API. Refresh tokens can be revoked; access tokens expire within their configured lifetime. Administrator corrections remain attributable.

8. Submission

The academic PDF contains editable identity placeholders in its source Markdown. Fill in college, student, roll number, supervisor and academic year, then rerun the packaging script. Include only measured test results. Keep attestation and real-device limitations visible in both the oral demonstration and report.

Deployment guide

Local prerequisites and compatibility

Use Python 3.12, `uv`, Node 24 LTS (or a compatible Node 22 version meeting the locked tool engines), Docker with Compose, and Git. On Apple Silicon, install native ARM64 Python/Node and Docker Desktop for Apple Silicon. PostgreSQL 16 has ARM64 images. OpenCV and MediaPipe wheels must match the interpreter and architecture; avoid mixing x86 Homebrew libraries with ARM64 wheels. This environment's measured platform is Linux x86_64, so macOS native validation remains a manual task.

The normal setup is `make setup`, `make db`, `make migrate`, `make seed`, followed by `make api` and `make dashboard`. The generated `.env` contains unique local secrets. Protect it with restrictive permissions and never include it in an academic submission. For phones, configure `mobile/.env` with a reachable API URL; it contains no credentials.

Linux hosts also need `libgl1`, `libgles2`, `libegl1` and `libglib2.0-0`. The Dockerfile installs these through the distribution package manager. In the restricted cloud workspace, `scripts/cloud_install.sh` extracts verified distribution libraries from a temporary container into ignored local runtime storage; `scripts/backend.sh` activates that location. It does not replace system libraries or disable package/TLS verification. On a normal Debian/Ubuntu machine, installing the packages with the system package manager is simpler.

Docker workflow

1. Generate `.env` and checksum-verified face models.

2. Run `docker compose up --build -d`.

3. Run `docker compose exec backend python -m app.seed` for local demonstration accounts only.

4. Check API `/health` and `/ready`; readiness must load all model assets.

5. Sign into the dashboard and configure real classroom data.

The backend image uses a pinned uv executable and a frozen Python lockfile. The web image uses `npm ci` and nginx. PostgreSQL has a health check and a persistent named volume. API migrations run before uvicorn at container startup. Persistent database files and model files survive process restarts; HTTP processes and authentication sessions in browser memory do not.

Corporate/cloud TLS interception may require the existing trusted CA bundle inside build steps. Both Dockerfiles support an optional BuildKit secret named `ca_bundle`. The cloud installer supplies the host's trusted public CA bundle using `--secret`; it is not baked into the image or included in artifacts. Normal direct-Internet Compose builds need no extra secret. Never use insecure TLS flags as a workaround.

Production-inspired deployment changes

The Compose configuration is a development baseline. Add trusted HTTPS termination, a non-demo administrator provisioning process, institution-specific CORS origins, encrypted backups, key rotation and a retention policy. Remove demo accounts or replace their passwords before real use. Keep the database port private. Use a separate least-privilege application database role and reviewed migration credentials; the development `spvs` role is also the database owner.

Verify `JWT_SECRET` and `BIOMETRIC_KEY` are independent, generated secrets delivered through a secret manager. Rotating the biometric key without re-encryption makes stored templates/device keys

unreadable. Do not invent credentials for provider attestation. Add a provider-backed verifier only with real Google/Apple project credentials and package/signing configuration.

Android development build

Install Android Studio and its supported SDK/NDK tools, enable USB debugging on a test phone and run `npm run android` in `mobile`. The generated project declares the exact SDK/Gradle versions selected by the locked Expo release. `expo prebuild --platform android --no-install` regenerates native code and discovers the local Wi-Fi module. Do not use Expo Go for that module. Gradle compilation, a Metro/Hermes export and physical device tests are distinct checks.

Use a trusted HTTPS URL for distribution. Development HTTP on a local network is only for controlled local testing. Do not put a raw API secret or biometric key in `EXPO_PUBLIC_*`: those values are embedded in the client bundle. iOS builds require Xcode/signing on macOS, and the application reports Wi-Fi scanning as unsupported.

Troubleshooting

Symptom	Diagnosis / action
Database connection failure	Start Compose database, confirm port availability and `.env` URL
Backend config validation fails	Generate real JWT/Fernet secrets; placeholders are intentionally invalid
Readiness returns 503	Run verified model downloader; inspect missing assets and writable caches
Official downloads denied	Allow `media.githubusercontent.com` and `storage.googleapis.com` in environment settings
Model checksum mismatch	Stop using the file; retry official source and investigate upstream provenance
Phone cannot reach API	Use LAN address/HTTPS; `localhost` on a phone points to the phone
Wi-Fi unavailable	Confirm native build, location permission/services and Android scan throttling; follow policy
Frequent liveness retries	Evaluate frame timing, slow blink, pose direction and lighting on the actual device
Browser session disappears on reload	Intended in-memory token storage; sign in again
Duplicate request conflicts	Preserve exact signed payload/key for a retry; use new challenge for new evidence

Do not delete the PostgreSQL volume to resolve ordinary setup issues. `docker compose down -v` is destructive and is not part of normal setup or restart instructions.

Android dependency download in this cloud

The observed Gradle build used Java 17, Gradle 9.4.1, Android API/build tools 37 and NDK 27.1.12297006. React Native 0.88.0’s AAR redirects from Maven Central to `repo.reactnative.dev`; allow that specific domain if the proxy returns HTTP 403. The saved cloud configuration includes this addition, but saving a draft does not apply it. Review/save the environment settings and publish before retrying. Keep TLS verification enabled, using the environment’s trusted Java CA store where necessary. Full native compilation and device execution remain unverified in the delivered results.

User manual

Administrator

Sign in using the dashboard. In Administration, create staff and student accounts with unique emails and student roll numbers. Assign the faculty account when creating a course, then enroll the correct students. The current role is enforced immediately by the backend; changing a role revokes refresh sessions and device binding and requires an explanation.

In Classrooms, enter surveyed coordinates and a suitable radius. A radius must be 5–300 m. Enter actual Wi-Fi BSSIDs and minimum RSSI values only after collecting a campus reference profile; each line is `aa:bb:cc:dd:ee:ff,-80`. Leave the list empty when unknown, then select a deliberate unavailable policy. The default is manual review. Do not label the demo coordinates as real campus measurements.

Device reset requests appear in Administration. Verify student identity before approving. An approval revokes the previous device; the student must then register the replacement phone. Policy changes are recorded with the old and new values. Requiring attestation while its provider is unconfigured results in review, not silent bypass.

Faculty

Courses lists assigned courses and enrolled students. Sessions opens a bounded attendance window for one course and classroom. Choose local start/end times; the API receives timezone-aware ISO timestamps. Session details refresh approximately every 15 seconds and show attempts separately from official records. Close a session to stop further automatic attendance.

Inspect reasons and risk indicators before reviewing an attempt. In Corrections & reviews, choose the session and student, enter PRESENT/ABSENT/EXCUSED and a meaningful reason. A correction is visibly sourced as CORRECTION and retains its previous status. Resolving a dispute records an explanation; it does not itself create attendance. Make a separate correction if justified. Reports downloads actual authorized records as CSV.

Student

Install the development app and sign in. First register the phone using account-password confirmation. Then open Face enrollment, read the retention explanation and grant consent. Follow the center, turn and slow-blink prompts in even lighting. The server rejects zero/multiple faces, weak samples or incomplete action sequences.

For an open session, tap Verify my attendance. The app checks eligibility/device status, captures a fresh sequence, obtains a current location and gathers supported Wi-Fi observations. It signs the payload and displays the server's result. Do not leave during capture. If a network submission fails, retry the same submission to avoid a new attempt. If the challenge expires or evidence fails, start a fresh challenge when permitted.

History shows official records and attempts separately. Request review of a disputed attempt with an explanation. A PENDING_REVIEW result is not confirmed presence. Settings opens OS permission controls and signs out. Enrollment allows password-confirmed template deletion; Device allows a replacement request. No background location collection is implemented.

Result interpretation

State	Meaning	Next action
PRESENT	All configured gates passed and attendance was recorded	Check history
REJECTED	A mandatory condition failed or a duplicate record exists	Inspect reason; request review if disputed
PENDING_REVIEW	Location/signals/attestation are uncertain under policy	Contact faculty; do not assume attendance approval
REQUIRES_RETRY	Fix a recoverable evidence issue	Enable permissions or obtain a better GPS fix, then retry
EXPIRED	Session/challenge is no longer valid	Ask faculty about a new session or review

Demonstration script

Before the evaluation

Run the automated suites and retain their current result files. Configure a real classroom and a deliberate Wi-Fi fallback. Complete the real-device checklist with consenting participants before promising a successful live capture. Prepare the randomly generated demo password locally; never place it in slides or the ZIP.

Eight-minute desktop demonstration

1. Explain the two-layer design and the reason uncertain signals produce review.

2. Sign in as administrator. Show the actual course, enrolled demo student and classroom configuration. Point out that initial metrics are real counts rather than seeded attendance.

3. Open a faculty session for the assigned course and a short future window.

4. Show the session page, separate attempts and official record tables, and automatic refresh.

5. Show the policy's manual-review Wi-Fi default and unconfigured attestation status.

6. Sign in as faculty and demonstrate that administrator navigation and API permissions are unavailable. An automated test also verifies cross-course denial.

7. Explain an authorized correction, enter a reason only for a legitimately reviewed record, and show its source/history. Do not misrepresent the correction as biometric verification.

8. Close the session and export the actual CSV. Open the security event log and identify session/correction/export events.

Phone demonstration after hardware validation

Sign in as a student, register the phone, consent to face enrollment and follow the randomized prompts. Open the active session, capture a fresh sequence, collect real location and available Wi-Fi, and show the exact server result. A review/retry result is a valid demonstration of the decision policy; never switch to fake evidence to force PRESENT.

Demonstrate one rejected or reviewable condition: permission denial, a clearly outside location, a revoked device, or a static-photo liveness failure. Then show the corresponding attempt, reason and dispute workflow. Any successful person-to-session result must come from actual captured evidence or be visibly identified as a faculty correction.

If hardware or network is unavailable

Demonstrate the running web/API/database workflow, real-model smoke tests and signed-request protocol tests. State that the API happy-path tests isolate vision and that physical sensor/liveness performance has not been measured. Show `SPVS_VERIFICATION_REPORT.md`, not an invented success video.

Test and build results

Generated from actual result files at **2026-10-23 03:02 IST**. JUnit timestamps reflect the Linux runner clock.

Suite	Passed	Failed/errors	Skipped	Execution platform
Backend / PostgreSQL (includes real-model tests)	72	0	0	Linux x86_64
Dashboard interactions	8	0	0	Linux x86_64
Mobile protocol and UI logic	7	0	0	Linux x86_64
Chromium against running containers	2	0	0	Linux x86_64

Unique suite totals: **89 passed, 0 failed/errors, 0 skipped**.

Commands executed

- `bash scripts/backend.sh pytest --junitxml=../artifacts/backend-tests.xml`
- `cd dashboard && npm test -- --run --reporter=junit --outputFile=../artifacts/dashboard-tests.xml`
- `cd mobile && npm test -- --run --reporter=junit --outputFile=../artifacts/mobile-tests.xml`
- `cd dashboard && npx playwright test`

Build and runtime validation

Capability	Observed outcome
Execution environment	Linux x86_64; Python 3.12, Node 24, PostgreSQL 16; no macOS ARM64 execution.
Reusable cloud setup	PASS: bash scripts/cloud_install.sh completed; npm ci, uv sync --frozen, three checksum-verified models, migrations, repeatable seed and both Docker builds. Existing local secrets preserved.
Database schema	PASS: alembic upgrade head and alembic check; no new upgrade operations detected; PostgreSQL integration tests passed.
Backend image and runtime	PASS: docker build with trusted CA secret; docker compose up -d --no-build db backend dashboard recreated services successfully. GET /health reports database connected; GET /ready reports face_models loaded and attestation not_configured.
Dashboard production build	PASS: npm run build (TypeScript and Vite); nginx Docker image starts and serves the actual API-connected dashboard.
Browser integration	PASS: npx playwright test against running Docker services; administrator room/session/export/close workflow and scoped faculty workflow. Tests use the created session ID and wait for navigation to remain repeatable.
Mobile type checking	PASS: npm run typecheck after the verification-result navigation correction.
Mobile Android bundle	PASS: CI=1 EXPO_NO_TELEMETRY=1 npm run export after the navigation correction; Metro/Hermes produced the actual Android bundle.
Android native preparation	PASS: Expo Android prebuild and local spvs-wifi module autolinking. Java 17, Gradle 9.4.1, Android API/build tools 37 and NDK 27.1.12297006 installed for the attempted build.
Android APK build	BLOCKED: ./gradlew :app:assembleDebug --no-daemon --max-workers=2 -PreactNativeArchitectures=arm64-v8a failed while resolving react-android-0.88.0-debug.aar. Maven Central redirects to repo.reactnative.dev, blocked by the cloud proxy with HTTP 403. Domain added to the saved environment draft; apply configuration before retrying. No APK produced; native compilation remains unverified.
Physical phone, iOS and Apple Silicon	NOT RUN: requires device/macOS hardware. Camera liveness, actual GPS/Wi-Fi behavior, accessibility and latency need the documented device matrix.
Platform attestation	UNCONFIGURED: adapter returns unverified; external Play Integrity/App Attest developer configuration is required.
Cloud configuration persistence	SAVED DRAFT: tested install/start instructions plus repo.reactnative.dev network addition. Saving is not applying/publishing; fresh-task restoration has not been independently tested.

Dependency audits

- Backend: 0 runtime packages with reported vulnerabilities in the locked-requirements audit.

- Dashboard: 0 affected package entries (0 critical, 0 high, 0 moderate, 0 low).

- Mobile: 45 affected package entries (0 critical, 45 high, 0 moderate, 0 low).

Audit commands: `npm audit --json` in each client; `pip-audit -r .runtime/backend-requirements.txt --format json` using requirements exported from `backend/uv.lock`. Raw reports are in `docs/evidence`.

Interpretation and unverified behavior

The backend's four real-model smoke cases are included in its total, not counted twice. Its API transaction happy paths explicitly substitute vision inside pytest fixtures; they do not prove live human capture. Real-model smoke tests execute the current YuNet, SFace and MediaPipe Tasks pipeline. No physical phone, iOS device, macOS ARM64 run, provider-backed attestation or population biometric evaluation was performed.

The mobile audit's affected entries propagate two root advisories through the Expo/Metro tree: GHSA-vfj7-8cjw-p6xm (`braces`) and GHSA-86w9-cpqp-85rv (`node-forge`). The registry reported no fixed version. See `KNOWN_LIMITATIONS.md`; do not interpret build success as security certification.

No manual sensor or live-biometric success is claimed. Intermediate failures included a missing graphics runtime, cloud CA trust configuration, older dependency advisories and test locator/assertion errors; these were diagnosed before the recorded final successful checks. Any still-blocked native capability appears in the build table.